

Formal Verification of the eBPF Verifier: Analysis of Three Papers and Capstone Research Directions

eBPF Capstone Research Assistant — Formal Research Memorandum

August 3, 2026

Contents

TL;DR	1
Key Findings	2
Details	4
Recommendations (staged, for a capstone)	6
Author Pages and Groups to Monitor	7
Caveats	7
References	8

TL;DR

- All three papers were located and read in full. They form a single, coherent research arc from the Rutgers Architecture and Programming Languages (RAPL) group: (1) **tnums/CGO 2022** formally specifies and proves the soundness/optimality of the eBPF verifier’s tristate-number arithmetic and contributes a new, faster, more precise multiplication algorithm now in the Linux kernel; (2) **Agni/CAV 2023** automatically verifies the *soundness* of the verifier’s entire range-analysis (five abstract domains) directly from kernel C source across 16 kernel versions; (3) **SAS 2025** shifts from soundness to *precision*, formally comparing competing abstract operators and synthesizing witness programs, yielding an upstreamed more-precise multiplication operator.
- The venues are all top-tier: CGO, CAV, and SAS are the premier venues in compiler optimization, computer-aided verification, and static analysis respectively (CAV 2025, for scale, accepted 79 papers from 305 submissions, ~25.9%). This is a strong, well-recognized line of work with direct, documented Linux-kernel impact.
- The clearest scope for a capstone lies in the verifier subsystems these papers explicitly leave unverified — path pruning/state-equivalence, register liveness, pointer/alignment tracking, speculative-execution (Spectre) mitigations, memory/stack safety, and the verifier–JIT interaction — plus practical precision/rejection tooling and keeping automated verification working against newer kernels and newer LLVM.

Key Findings

The three papers, precisely

1. **“Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers”** — Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, Santosh Nagarakatte. CGO 2022 (20th IEEE/ACM Int’l Symposium on Code Generation and Optimization), pp. 254–265, DOI 10.1109/CGO53902.2022.9741267. Extended version: arXiv:2105.05398. This is the “tnum paper.” It provides the *first* formal specification of the tnum (tristate number) abstract domain and the first paper-and-pen proofs of **soundness and optimality (maximal precision)** of the kernel’s $O(1)$ tnum addition and subtraction operators, plus a **new, provably sound tnum multiplication algorithm**. Per the paper, the new multiplication “takes 262 cycles” on average versus “around 393 cycles” for the kernel’s prior `kern_mul` — “our_mul 33% (resp. 32%) faster on average than `kern_mul` (resp. our optimized version of `bitwise_mul`)” — and produces a more precise tnum in most ($\approx 80\%$ of 8-bit) differing input pairs. This new multiplication was upstreamed into the Linux kernel.
2. **“Verifying the Verifier: eBPF Range Analysis Verification” (Agni)** — Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, Santosh Nagarakatte. CAV 2023 (35th Int’l Conference on Computer Aided Verification), LNCS 13966, pp. 226–251, DOI 10.1007/978-3-031-37709-9_12 (Open Access). Automatically checks the **soundness** of the verifier’s value tracking (range analysis) across all five abstract domains (u64, u32, s64, s32 intervals + tnum). It automatically generates SMT verification conditions directly from the kernel’s C source, checks a general soundness specification (and a refined one that preconditions inputs using the verifier’s “shared refinement/suffix” operator), and — when a proof fails — uses **differential synthesis** to generate a concrete eBPF program (a proof-of-concept) that manifests the unsoundness. It checked 16 kernel versions (4.14–5.19), found 27 previously unknown bugs, proved recent kernels sound, and generated POCs for $\sim 97\%$ of soundness violations.
3. **“Comparing the Precision of Abstract Operators in the eBPF Verifier using Differential Synthesis”** — Matan Shachnai, Harishankar Vishwanathan, Srinivas Narayana, Santosh Nagarakatte. SAS 2025 (32nd Static Analysis Symposium), Singapore, DOI 10.1007/978-3-032-07106-4_7. A follow-up to Agni that pivots from soundness to **precision**. It provides a formal precision preorder over abstract operators, uses the Agni C-to-logic tooling to compare two competing operator implementations, and adapts differential synthesis to generate **witness programs** demonstrating that one operator is strictly more precise in a real verifier execution. It produced a more precise `bpf_mul` operator (upstreamed), evaluated precision-related kernel patches (v5.7–v6.8), and empirically demonstrated the value of the verifier’s reduction operator (`BPF_SYNC`).

The research arc (chronological and technical)

The three papers are tightly linked and build on each other:

- **CGO 2022 (tnums)** establishes the *formal foundations for a single domain* — the tnum domain — with hand proofs and bounded SMT verification. It found kernel tnum multiplication resisted proof, motivating a new algorithm. This is the narrowest and deepest of the three (one domain, analytical proofs, unbounded-bitwidth soundness/optimality).
- **CAV 2023 (Agni)** *generalizes to the whole value-tracking analysis* — all five domains and their non-modular composition — and automates the process by extracting verification conditions from kernel C code, so it can track the fast-moving kernel. Its key conceptual contribution is a soundness specifi-

cation for **abstraction/reduction operators** that mix abstraction and cross-domain reduction “in one shot,” plus differential synthesis for POC generation.

- **SAS 2024 (bridge paper)** — “Fixing Latent Unsound Abstract Operators in the eBPF Verifier of the Linux Kernel” (Shachnai, Vishwanathan, Narayana, Nagarakatte), SAS 2024, LNCS 14995, pp. 386–406, DOI 10.1007/978-3-031-74776-2_15 — is an intermediate NEAT/experience paper (not in the user’s list but central to the arc). It reports that several abstract operators are **unsound in isolation** but corrected by the shared refinement operator (“latent” unsoundness), and that the team re-engineered Agni with a divide-and-conquer approach (in collaboration with kernel developers, notably Paul Chaignon) to scale verification to kernels through v6.8 after Andrii Nakryiko’s refinement-operator patch caused Agni timeouts. Their patches removing latent unsoundness were upstreamed.
- **SAS 2025 (precision)** *reuses the same C-to-logic + differential-synthesis machinery* but redefines the target property from soundness to precision. This closes the loop: the group first proved the verifier is *sound* (won’t accept unsafe programs), then attacked the *precision* problem (the verifier rejecting safe programs), which is the dominant practical pain point for eBPF developers.
- **Beyond (2025, context)**: The group’s newest work — “Automatic Synthesis of Abstract Operators for eBPF” (eBPF’25 SIGCOMM workshop), tool named **Vayu** — uses CEGIS to *automatically synthesize* sound and precise operators, and Narayana’s 2025 LSFMM+BPF session extends Agni toward **path-pruning** verification. These indicate the direction the group is heading.

Venue prestige (confirmed)

- **CAV** (Computer Aided Verification): the premier international conference on computer-aided/formal verification. Highly selective — CAV 2025 selected 79 papers (51 regular, 24 tool, 4 case-study) from 305 submissions (~25.9%). Springer LNCS proceedings.
- **CGO** (Code Generation and Optimization): the premier venue at the hardware/software interface for compiler optimization and code generation (IEEE/ACM).
- **SAS** (Static Analysis Symposium): the leading venue dedicated specifically to static analysis and abstract interpretation (Springer LNCS).
- All three are highly selective, archival, peer-reviewed PL/verification venues. The choice of venue tracks the papers’ emphasis: CGO (a new algorithm + performance), CAV (automated verification), SAS (abstract-interpretation precision theory).

Documented Linux-kernel impact

- **tnum multiplication** (CGO 2022): upstreamed as commit 05924717ac70 (“bpf, tnums: Provably sound, faster, and more precise algorithm for tnum_mul”), applied to bpf-next around June 1, 2021, shipping in **Linux 5.14** (the authors describe it as upstreamed in May 2021; it modifies kernel/bpf/tnum.c).
- **BPF_MUL precision** (SAS 2025): upstreamed as commit 9aa0ebde0014 (“bpf, verifier: Improve precision of BPF_MUL,” author Matan Shachnai), applied to bpf-next Dec 30, 2024, shipping in **Linux 6.14**; the commit message notes soundness proved via Non-Linear Integer Arithmetic and via Agni at 8-bit bitwidths.
- **Latent unsoundness fixes** (SAS 2024): patches hardening and/or/xor value tracking upstreamed (e.g., commit 1f586614f3ff).
- **BPF_ADD/BPF_SUB precision** (Vayu, eBPF’25): upstreamed as commit 7a998a731627 in June 2025 (likely Linux 6.17 — see Caveats).

Details

Technical substance of each paper

tnums / CGO 2022. The tnum domain tracks, per bit, whether it is known-0, known-1, or unknown (μ), implemented in the kernel as a pair (`value`, `mask`). The paper formalizes the abstraction α and concretization γ functions, shows they form a Galois connection, and proves the kernel’s `tnum_add/tnum_sub` are both **sound and optimal** (equal to α -compose- f -compose- γ , i.e., abstraction composed with the concrete operation composed with concretization) despite running in $O(1)$ rather than the $O(n)$ ripple-carry one might expect — the crux is the “minimum/maximum carries” lemmas showing which carry positions are forced vs. uncertain. The authors could not prove the kernel’s multiplication (`kern_mul`) sound analytically (bounded SMT verification succeeds only to 8 bits; times out beyond 16 bits due to nonlinear ops and unrolled loops), so they designed `our_mul`: a single loop over the bitwidth using a “value-mask decomposition,” provably sound for unbounded bitwidth, more precise, and faster (262 cycles avg vs. 393 for `kern_mul`, i.e., 33% faster). They note `our_mul` is sound but *not optimal* and pose open questions on a sound+optimal+fast multiplication.

Agni / CAV 2023. The central difficulty is that the kernel does *not* use a textbook modular reduced product; it interleaves abstraction with cross-domain reduction (“abstraction/reduction operators”), so each domain’s individual correctness does not imply their composition’s correctness. Agni therefore reasons “in one shot.” Its pipeline: compile `verifier.c` to LLVM IR; slice per-operator via injected opcode + dead-code elimination; model memory with a bitvector-tree using LLVM MemorySSA; emit SMT-LIB (bitvector theory); check with Z3. The general soundness spec (Eq. 2) over-approximates reachable inputs and flags many “latent” violations; a refined spec (Eq. 4) preconditions inputs with the kernel’s shared reduction “suffix” (`reg_bounds_sync`), which proves recent kernels sound. Differential synthesis enumerates loop-free eBPF programs of increasing length and uses an existentially-quantified SMT query to synthesize a POC. Results: 16 kernel versions (4.14–5.19), 27 new bugs, POCs for ~97% of violations (≤ 3 instructions in most cases), all operators sound for kernels newer than 5.13 under the refined spec. **Excludes multiplication** (SMT timeout with 64-bit bitvector multiply; verified at 8-bit).

Precision / SAS 2025. Defines a precision preorder $F1 \# \leq F2$ (“ $F1$ at least as precise as $F2$ for all inputs”) over the product lattice of the five domains, checkable by SMT, plus a strict-precision existential query per domain. Crucially it distinguishes **valid** abstract inputs from **reachable** ones — the verifier starts every register as known-singleton or fully-unknown (top element of the lattice) and evolves via abstract semantics + reduction, so many valid inputs are unreachable. Hence a proved precision gain may never manifest, motivating witness synthesis (adapted from Agni’s differential synthesis). The paper presents a new, more precise `bpf_mul` (handles negative/signed operands and full u64 ranges via interval arithmetic instead of widening to the top element), proves it is at least as precise as the old operator, and synthesizes a witness. It evaluates precision patches across v5.7–v6.8 (BPF_AND, BPF_SYNC, BPF_XOR, BPF_MUL), generating witnesses for all, and empirically shows the reduction operator (BPF_SYNC) improves precision across nearly all domains (RQ3), though it could not synthesize tnum-domain witnesses for bitwise/shift ops within the length/time bounds — an acknowledged limitation of bounded enumeration.

Open problems explicitly flagged by the authors

- **Only range/value tracking is covered.** Agni’s Limitations section states plainly that “there are other static analyses in the kernel verifier beyond range analysis... These include tracking register liveness for reading and writing, and detecting speculative execution vulnerabilities.” These remain unverified.
- **Multiplication coverage gap.** Agni excludes `mul` (64-bit bitvector multiply causes SMT timeouts; verified only at 8-bit). CGO 2022’s `our_mul` is sound but **not optimal**, and the authors pose explicit

open questions: (1) capturing correlated unknown bits across partial products, (2) a sound/precise/fast n -ary tnum addition, (3) using concrete multiplication over masks.

- **Trusted computing base.** Agni’s proofs depend on LLVM, Z3, and the team’s translation passes; bugs there are possible.
- **Differential synthesis incompleteness.** Enumeration times out for programs of length ≥ 4 ; a non-manifested violation may be latent or require longer programs. SAS 2025 inherits this (bounded $L=4$; could not expose tnum-domain precision gaps for some ops).
- **Scalability / maintainability.** The SAS 2024 bridge paper and the 2025 LWN/LSFMM coverage document that Agni’s verification time exploded (2.5 h for 4.14 $\rightarrow$ ~ 10 h for 5.13 $\rightarrow$ weeks for 6.4; timeouts after Nakryiko’s refinement patch), forcing a divide-and-conquer re-engineering. A dependency on LLVM type information not present in LLVM ≥ 15 threatens to break Agni’s C-to-SMT translation on future kernels.

Gaps between what is verified and what remains unverified

Verified so far (by this group): soundness of value tracking / range analysis across five domains (kernels ≥ 5.13); soundness+optimality of tnum add/sub; a sound new tnum mul; precision relations of selected arithmetic/bitwise operators and the reduction operator.

Unverified in the eBPF verifier:

- register/stack **liveness tracking**;
- **path pruning / state-equivalence** (`states_equal/regsafe`) — in progress by the group but not published as a completed verification;
- **pointer arithmetic and type tracking** (`PTR_TO_CTX`, `PTR_TO_MAP_VALUE`, etc.);
- **alignment tracking**;
- **speculative-execution (Spectre) mitigations**;
- **stack/memory safety and helper-argument checks**;
- **program-termination/complexity limits**;
- the **JIT compilers** (a separate line — Jitterbug — verifies BPF JITs but assumes the verifier is correct, and no work connects verifier and JIT correctness end-to-end);
- interactions between the verifier and the JIT.

Related and competing work (field context)

- **PREVAIL** (Gershuni et al., PLDI 2019): an *alternative offline* eBPF verifier built on abstract interpretation with the **Zone** relational domain; supports loops; underlies the **Windows** eBPF verifier. It is a different design point (offline, richer domains, over-approximates bitwise ops) rather than a verification of the Linux verifier. SeaHorn/Crab underlie PREVAIL’s implementation.
- **Bhat & Shacham (2022)**, “Formal Verification of the Linux Kernel eBPF Verifier Range Analysis” (UT Austin manuscript): the closest concurrent effort to Agni, also verifying range-analysis soundness from C source; Agni distinguishes itself by exposing the non-modular structure, handling branches, auto-generating VCs, and synthesizing POCs.
- **Jitterbug / Serval** (Nelson, Van Geffen, Torlak, Wang; OSDI 2020 / SOSP 2019, UW UNSAT group): push-button verification of BPF **JIT compilers**. Per the OSDI 2020 paper they “designed, implemented, and verified a new BPF JIT for 32-bit RISC-V, found and fixed 16 previously unknown bugs in five other deployed JITs,” all upstreamed; the JIT-correctness specification catches 80 of 82 known JIT bugs (missing only two in offset-table construction). Complementary — it verifies the JIT and *assumes* the verifier is correct.

- **Hao Sun & Zhendong Su (ETH Zürich):** “Validating the eBPF Verifier via State Embedding” (OSDI 2024) — a dynamic/testing approach embedding concrete states as self-checks; within one month it “uncovered 15 previously unknown logic bugs, 10 of which have already been fixed,” of which two are exploitable and can lead to local privilege escalation. Also “Finding Correctness Bugs in eBPF Verifier with Structured and Sanitized Program” (EuroSys 2024). Testing/fuzzing rather than formal proof.
- **Onderka & Ratschan (VMCAI 2022),** “Fast Three-Valued Abstract Bit-Vector Arithmetic”: independent, optimal polynomial-time three-valued (tnum-like) arithmetic — directly relevant alternative algorithms for the tnum domain.
- **Abstract-transformer synthesis:** Amurth (Kalita et al., OOPSLA 2022) and its reduced-product extension (SAS 2025) synthesize sound+precise abstract transformers; the RAPL group’s Vayu applies this idea to eBPF.
- **Practical precision/usability & fuzzing:** PREVAIL precision studies; BRF fuzzing (Hung & Amiri Sani, FSE 2024); bpfchecker differential fuzzing (CCS 2024); SpecCheck/specification-based fuzzing (Waterloo); “Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections” (Zheng et al., 2026) — empirically, 47% of rejections return only EINVAL and one error string maps to up to nine root causes; language-verifier-gap work (Rex/Rax, SafeBPF, ePass). These connect formal precision results to the real-world “verifier rejects safe programs” pain.

Recommendations (staged, for a capstone)

Stage 0 — Reproduce and orient (weeks 1–4). Clone and run **Agni** (github.com/bpfverif/agni) and the tnum artifacts (github.com/bpfverif/tnums-cgo22); reproduce soundness checking on one or two kernel versions and one differential-synthesis POC. Deliverable: a working pipeline and a written explanation of the $C \rightarrow \text{LLVM IR} \rightarrow \text{SMT}$ flow. *Threshold to proceed:* you can regenerate a known POC and a soundness proof for at least one operator.

Stage 1 — Pick exactly one unverified subsystem and scope a specification (weeks 4–8). The highest-value, best-defined targets, in rough order of tractability for a capstone:

1. **Extend Agni to a newer kernel and/or newer LLVM.** Concrete, high-impact, and explicitly needed: the LLVM ≥ 15 type-information problem and post-v6.8 refinement changes. Deliverable: Agni running on a current LTS kernel. *Benchmark that changes the plan:* if the LLVMToSMT rewrite proves larger than one term, narrow to a single operator family.
2. **Multiplication precision/optimality.** Attempt a sound *and optimal* tnum multiplication (CGO 2022’s open questions 1–3), or extend Agni’s soundness check to mul using non-linear arithmetic (NIA) solvers or bit-width induction rather than bit-blasting. Deliverable: a proof or a solver strategy that scales past 8-bit.
3. **Precision-to-rejection bridge.** Combine the SAS 2025 precision framework with the rejection-diagnosis literature (Zheng et al. 2026): take real rejected-but-safe programs (Cilium/Katran/Aya workaround commits) and test whether a synthesized more-precise operator would have accepted them. Deliverable: a quantified link between operator imprecision and real rejections — novel and practically compelling.

Stage 2 — Verify and synthesize (weeks 8–13). Prove your specification with Z3 (reuse Agni’s VC generator), and — if targeting a new subsystem (liveness or a slice of pointer/alignment tracking) — write the soundness specification first, then attempt automated checking. Use differential/witness synthesis to produce concrete eBPF programs as evidence. *Threshold:* at least one machine-checked result or one synthesized witness/POC.

Stage 3 — Upstream-quality artifact (weeks 13–15). Package a reproducible artifact and, if you produced an operator improvement or bug, draft a kernel patch with an auto-generated witness in the commit message (the group’s demonstrated path to impact). Even a negative result (a subsystem that resists tractable specification) is a publishable capstone finding.

Concrete direction picks by interest:

- *Theory-leaning*: sound+optimal fast tnum multiplication; formalizing the reduction operator as a proper Granger reduction.
- *Systems/tooling-leaning*: port Agni to current kernels/LLVM; CI integration so kernel patches are checked at review time (the authors’ stated goal).
- *Security-leaning*: specify and check a slice of pointer/alignment or Spectre-mitigation logic; or reproduce/extend state-embedding-style differential checks and compare against Agni’s formal results.
- *Usability-leaning*: precision-to-rejection bridge connecting SAS 2025 to real rejection datasets.

Author Pages and Groups to Monitor

- **Rutgers Architecture and Programming Languages (RAPL) group** — people.cs.rutgers.edu/~sn349/rapl/; the Agni project page people.cs.rutgers.edu/~sn349/agni/ and github.com/bpfverif (Agni, tnums-cgo22, Vayu).
- **Santosh Nagarakatte** — people.cs.rutgers.edu/~santosh.nagarakatte/ (PI; also RLIBM, SoftBound-CETS).
- **Srinivas Narayana** — people.cs.rutgers.edu/~sn624/ (PI; leads path-pruning verification).
- **Harishankar Vishwanathan** — harishankarv.github.io (lead student author; upstream patches).
- **Matan Shachnai** — mshachnai.github.io (lead author SAS 2024/2025; now University of Arizona).
- **Competing/complementary groups**: UW UNSAT group (Luke Nelson, Xi Wang, Emina Torlak — Jitterbug/Serval); Hao Sun & Zhendong Su at ETH Zürich (state embedding, verifier fuzzing); VMware/Tel Aviv PREVAIL authors (Gershuni, Gurfinkel, Navas, Sagiv); Reps/Roy/D’Antoni (Amurth abstract-transformer synthesis); Jan Onderka & Stefan Ratschan (three-valued bit-vector arithmetic); Paul Chaignon (kernel-side collaborator on Agni). Prof. Ruzica Piskac (Yale) does not appear as a co-author on any of these eBPF-verifier papers; John Sarracino likewise does not appear in the author lists — the actual co-author set across all three papers is exactly Vishwanathan, Shachnai, Narayana, and Nagarakatte.
- **Venues/summits to watch**: SIGCOMM eBPF workshop, SAS, CAV, PLDI, OSDI, and the LSFMM+BPF Summit (LWN coverage), plus the eBPF Foundation research-grant announcements.

Caveats

- All three primary papers, plus the SAS 2024 bridge paper, were accessed in full (two via the user’s URLs; the tnum paper via arXiv:2105.05398; SAS 2024 via the Rutgers preprint and Springer). Where quantitative claims (bug counts, cycle counts, precision percentages, kernel-version ranges) are quoted, these are stated directly by the authors in those papers.
- Exact kernel release numbers for upstreamed patches are corroborated from the authors’ home-pages, kernel mailing-list/patchwork archives, and LWN, not always from git.kernel.org directly (which blocked automated fetch). Commit IDs (05924717ac70, 9aa0ebde0014, 7a998a731627, 1f586614f3ff) are confirmed via multiple secondary sources. The Linux 6.17 attribution for the BPF_ADD/BPF_SUB (Vayu) patch is an inference from the June 2025 apply date relative to the 6.16 merge window and is flagged as unverified; the June 2025 upstreaming itself is documented.

- The “research arc” interpretation (how the three papers relate) synthesizes the papers’ own related-work/self-citation statements with the group’s website timeline and LWN/LSFMM reporting; the papers themselves explicitly cross-cite in this order, so the arc is well-supported rather than speculative.
- The path-pruning verification and Vayu operator-synthesis efforts are recent (2025) and, per LWN/LSFMM coverage, partly work-in-progress; these are flagged as ongoing rather than completed verification results.
- Acceptance-rate figures (CAV 2025: 79/305 $\approx$ 25.9%) come from the Springer proceedings and conference-statistics aggregators and vary year to year; they support the “top-tier/selective” characterization but should not be treated as exact for any specific year of publication.
- One correction of note: the UW Jitterbug work’s canonical figure is **16** previously unknown JIT bugs found and fixed (from the OSDI 2020 paper); some secondary slide decks cite “30+” counting all patches/optimizations, but the peer-reviewed number is 16.

References

1. Vishwanathan, H., Shachnai, M., Narayana, S., & Nagarakatte, S. (2022). Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers. *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 254–265. DOI: 10.1109/CGO53902.2022.9741267. Extended version: arXiv:2105.05398.
2. Vishwanathan, H., Shachnai, M., Narayana, S., & Nagarakatte, S. (2023). Verifying the Verifier: eBPF Range Analysis Verification. *35th International Conference on Computer Aided Verification (CAV 2023)*, LNCS 13966, 226–251. DOI: 10.1007/978-3-031-37709-9_12.
3. Shachnai, M., Vishwanathan, H., Narayana, S., & Nagarakatte, S. (2025). Comparing the Precision of Abstract Operators in the eBPF Verifier Using Differential Synthesis. *32nd Static Analysis Symposium (SAS 2025)*. DOI: 10.1007/978-3-032-07106-4_7.
4. Shachnai, M., Vishwanathan, H., Narayana, S., & Nagarakatte, S. (2024). Fixing Latent Unsound Abstract Operators in the eBPF Verifier of the Linux Kernel. *31st Static Analysis Symposium (SAS 2024)*, LNCS 14995, 386–406. DOI: 10.1007/978-3-031-74776-2_15.
5. Gershuni, E., Amit, N., Gurfinkel, A., Narodytska, N., Navas, J. A., Rinetzky, N., Ryzhyk, L., & Sagiv, M. (2019). Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. *PLDI 2019*.
6. Nelson, L., Van Geffen, J., Torlak, E., & Wang, X. (2020). Specification and Verification in the Field: Applying Formal Methods to BPF Just-in-Time Compilers in the Linux Kernel. *OSDI 2020*.
7. Sun, H., & Su, Z. (2024). Validating the eBPF Verifier via State Embedding. *OSDI 2024*.
8. Onderka, J., & Ratschan, S. (2022). Fast Three-Valued Abstract Bit-Vector Arithmetic. *VMCAI 2022*.
9. Vishwanathan, H., et al. (2025). Automatic Synthesis of Abstract Operators for eBPF (Vayu). *3rd Workshop on eBPF and Kernel Extensions (eBPF ’25)*. DOI: 10.1145/3748355.3748361.
10. Bhat, S., & Shacham, H. (2022). Formal Verification of the Linux Kernel eBPF Verifier Range Analysis. Manuscript, UT Austin.